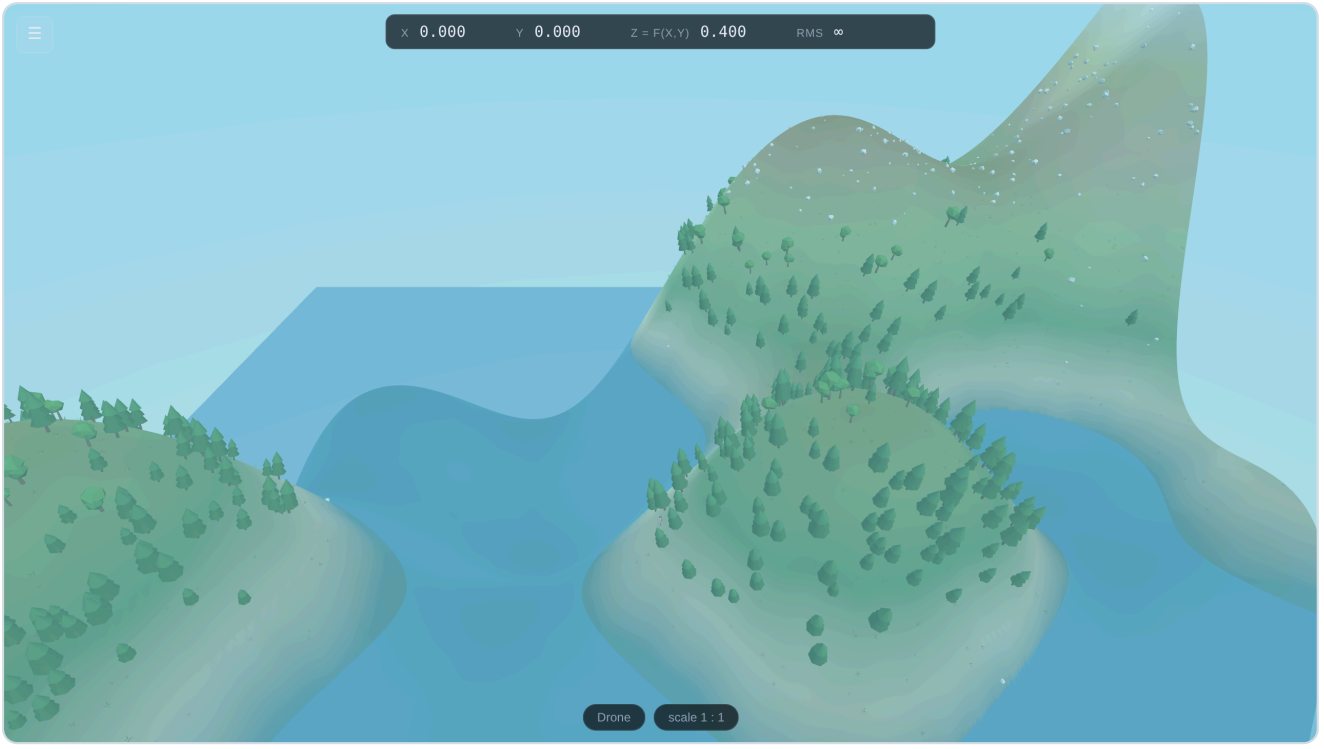
TEACHING GUIDE & USER MANUAL

Gradient Peaks

A 3D sandbox for surface plots of functions of two variables — walk on the graph of $f(x, y)$, read its partial derivatives off the ground, and find the maximum over a feasible set.



Written for someone who has never installed or run a program like this before.
Every step is spelled out. Nothing here requires you to write code.

Version 1.0 · Runs on Windows, macOS, Linux, iPad, iPhone and Android · No installation required

What is in this guide

1 Start here — the sixty-second version

You have been given a folder called `Gradient-Peaks`. If it arrived as a `.zip` file, you must **unzip it first** — see the box below. Inside are these items:

ITEM	WHAT IT IS
`Gradient-Peaks.html`	**The program.** Double-click it and it opens in your web browser and runs. This one file contains everything. It does not need the internet.
`Gradient-Peaks-Guide.pdf`	This document.
`website/`	A folder holding the same program split into its separate parts. You only need this if you want to *publish it on the internet* for your students (Part 11) or install it as an app (Part 12).
`README.txt`	A one-page plain-text summary of what you are reading now.

> Unzip it first — do not run it from inside the zip
> Double-clicking a `.zip` file often shows you the contents without actually extracting them, and programs run from that preview will misbehave.
>
> **Windows:** right-click the `.zip` → *Extract All…* → *Extract*.
> **macOS:** double-click the `.zip`. A normal folder appears next to it. Use that folder.

Open it

1. **Double-click `Gradient-Peaks.html`.**
2. Your web browser opens. You will see a dark screen with a small spinning circle and the words *Building the terrain…* for one to three seconds.
3. A green and grey mountain appears, seen from the air, with a control panel down the left-hand side. **That is it. It is running.**

There is nothing to install, no account to create, and no internet connection needed. The program is not "downloading" anything — it is drawing the mountain from the formula `f(x,y) = (x*y)^0.5`, which you can see typed in the box at the top of the panel.

> English and Spanish
> The program follows your browser: if that is set to Spanish, it opens in Spanish. You can switch at any time with the **English / Español** selector in the top corner of the control panel, without losing whatever you are in the middle of.
>
> If you publish it to a web address (Part 11), adding `?lang=es` to the end of the link makes it open in Spanish for your students whatever their browser is set to — and `?lang=en` forces English. **A Spanish edition of this guide is included as `Gradient-Peaks-Guia.pdf`.**

> If it opens in a browser you do not like
> Any modern browser works, but **Google Chrome** and **Microsoft Edge** give the smoothest result. To choose one: right-click `Gradient-Peaks.html` → *Open with* → pick your browser. On macOS the menu item is the same; hold `Control` and click if you have no right mouse button.

> One thing that will not work, and why
>
> Inside the `website/` folder there is another file called `index.html`. **Double-clicking that one gives you a blank screen.** That is not a fault. Browsers refuse, for security reasons, to let a page opened directly from your hard drive load its own separate program files. The single `Gradient-Peaks.html` avoids the problem by having everything already inside it.
>
> Use `Gradient-Peaks.html` on your own machine. Use the `website/` folder only when you publish it to a web address (Part 11).

2 Will it run on my computer?

Almost certainly yes. The program needs a graphics feature called **WebGL**, which every browser has had since about 2013.

YOU NEED	DETAILS
A computer or phone	Roughly 2014 or later. It has been kept deliberately small — under one megabyte — so it runs on school laptops and cheap tablets.
A modern browser	Chrome, Edge, Firefox or Safari, updated within the last few years.
Nothing else	No internet, no Java, no Flash, no plug-in, no administrator password.

Checking in ten seconds

Open `Gradient-Peaks.html`. If you see a mountain, you are done — that *is* the test. If you see a blank or black screen instead, go to Part 13.

> A note about school and university computers
>
> Because this is an ordinary web page, it does not trigger the software-installation restrictions that lock down most institutional machines. There is no installer and no `.exe`. If your students can open a web page, they can run this.

3 Your first five minutes

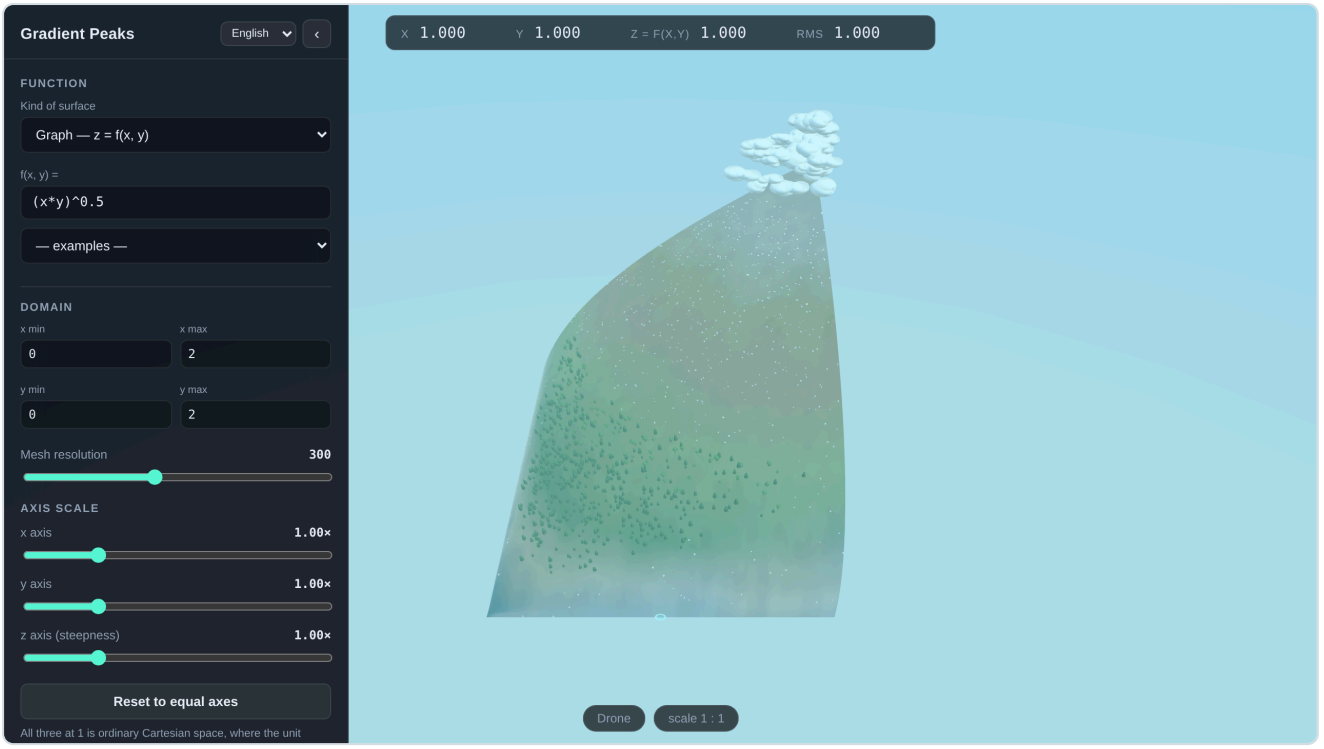


Figure 1. The opening screen. The control panel is on the left. Along the top, the readout always shows where you are: `x`, `y`, the height `z = f(x,y)` and `RMS`. At the bottom, the current view mode and scale. You begin in the air, looking at the whole surface.

The four numbers along the top

`x` and `y` are where the explorer is standing; `z = f(x,y)` is the height there. The fourth, **`RMS`**, is the absolute slope of the level curve through that point:

$$\text{RMS} = \left|\partial f/\partial x \div \partial f/\partial y\right|$$

Differentiating `f(x, y) = c` implicitly gives $dy/dx = -f_x/f_y$ along the contour, so this number says how many units of `y` one unit of `x` buys you while staying at the same height — the marginal rate of substitution. It is shown unsigned, it is there whether or not any derivative arrow is switched on, and it goes to `∞` exactly where the level curve turns vertical ($f_y = 0$). At a critical point, where both partials vanish, it is genuinely undefined and shows a dash.

Step one: look at the whole surface

You start as a **drone**, hovering above the plot. This first view is deliberate: before you go down and stand on the surface, you should see it as what it is — the graph of a function.

Step two: take control of the mouse

Click once anywhere on the mountain. The message *"Click to look around"* disappears and your mouse pointer vanishes. This is normal and is exactly how a 3D game behaves: your mouse now turns your head instead of moving a pointer.

Move the mouse to look around. Press `W` `A` `S` `D` to fly. Press `Space` to rise and `Ctrl` to descend. Hold `Shift` to go faster.

> The single most important key
> `Esc` gives you your mouse pointer back. If you ever feel trapped, press `Esc`. Nothing is lost. Click the scene again to resume looking around.

Step three: go and stand on it

Press `2`. You drop onto the surface behind a small orange-jacketed explorer, who is exactly **1.80 metres tall**. That figure is the ruler for everything else in the scene: the trees, the boulders, and the measuring circle you will meet in a moment are all sized against a person.

Walk with `W` `A` `S` `D`. Watch the numbers along the top change as you move. **That readout is the whole point of walking**: the student feels the slope in their hands while watching `z` climb.

Press `1` to see through the explorer's own eyes, `3` to go back to the drone, and `2` to return to the over-the-shoulder view.

Zooming, and the two things it could mean

"Zoom in" is two different requests, and they used to share one control.

The wheel moves the camera. Scroll up to come closer, down to pull back; on a trackpad, pinch. Nothing about the mathematics changes — the explorer is exactly the size they were, and so is every measurement made against them. Where the camera is somebody's own eyes and there is nowhere nearer to be, the wheel puts a longer lens on instead.

The wheel with `⌘` held resizes the explorer (`⊞` on Windows; `Alt` or `Option` works everywhere, and is the one to reach for if your system swallows the other). This is the zoom-in ruler of Part 4 under a different hand: shrinking the explorer magnifies the surface, and it is how you get down to the scale where a differentiable function is indistinguishable from its tangent plane.

Both jobs also have a **dial on the right edge of the screen**, because a tablet has no wheel and no modifier key at all. The upper dial is the camera, the lower one the explorer's size, and each has a small button above it that returns it to normal. They run the only direction a vertical dial can run: up is more of whatever it is labelled with.

The explorer's size runs from **1 : 10,000** — 0.18 mm tall, where a differentiable function is indistinguishable from its tangent plane — to **100 : 1**, where they are 180 m tall and cross a hillside at a stride. Both dials count *decades* rather than metres, which is the only way six orders of magnitude fit on one slider: a linear dial would spend nine tenths of its travel inside the largest decade and leave the interesting end unreachable.

Looking around without a mouse

Hold `⌘`/`⊞` or `Alt` and the movement keys stop moving you and start aiming you instead: `W` `A` `S` `D` or the arrow keys swing the view, at about a quarter turn a second, so a full look round takes four seconds and a small correction is a tap. It is clamped short of vertical, so the view never rolls under the ground it is standing on. This is for anyone working on a laptop without a trackpad they can aim with, or demonstrating at a board where a mouse is not to hand.

Held, not tapped — the way a 3D game does it. The mode lasts exactly as long as the key is down and ends the instant it comes up, and any key that was down at that moment is let go of too. That last part matters more than it sounds: while ⌘ is held, macOS does not deliver the release of an ordinary key at all, so a program that did not let go on its own would think you were still holding W and set off walking the moment you released ⌘.

> Which modifier, on which machine
>
> All three work, but the operating system reserves some combinations before any web page sees them. On a Mac, ⌘+W closes the tab and nothing a page does can stop it — so use ⌥ (Option) there if you want W A S D; ⌘ with the arrow keys is fine. On Windows, ⊞ +arrows is window snapping and never reaches the browser, so use ⊞ with W A S D, or Alt with either. Alt/⌥ is the one that works everywhere with everything.
>
> Rather than ask you to remember that, **Key to hold for looking** in the View section lets you set it: either key, Alt/⌥ only, or ⌘/⊞ only. It starts on *either*, which is the widest thing that can be offered without knowing whose keyboard this is, and it is remembered on the machine — so lose a tab once to ⌘+W and you can switch to Option and never think about it again. The same key is the one that puts the wheel on the explorer's size, so there is only ever one to remember.

Step four: get the panel back

Press Esc to release the mouse, then press Tab to hide or show the control panel. You can also click the ‹ button in its top corner, or the ☰ button that replaces it.

> Keyboard shortcuts pause while you type
>
> When your cursor is inside a text box, the letter keys type letters — they do not trigger shortcuts. So typing `x*y` will not accidentally toggle three settings. Press Enter when you have finished typing to apply what you wrote and leave the box.

4 The control panel, one control at a time

Language

The selector beside the panel title switches the whole interface between **English** and **Español** — including the error messages the parser produces. Nothing is recomputed, so you can switch mid-demonstration.

Function

Kind of surface — three of them, and only the first is a landscape you can walk on.

- **Graph — z = f(x, y)**: the default, and what the rest of this guide is about. One height above every point of the plane, so it can be stood on, differentiated and contoured.
- **Implicit — F(x, y, z) = 0**: everything satisfying one equation in three variables — a sphere, a torus, a cone. There may be several sheets above one `(x, y)`, or none, so there is no single-valued `∂f/∂x` — but the surface itself can absolutely be walked on, and Part 8 covers what "derivative" and "grid" mean once there is no `f` to have them.
- **Parametric — r(u, v)**: a sheet swept out by two parameters. Walked on the same way, in its own coordinates.

`f(x, y) =` — the formula that becomes the landscape. Type a new one and press `Enter`. Part 5 explains the notation in full.

— examples — — a menu of ready-made functions. Choosing one loads it and sets a sensible domain. This is the fastest way to explore. One of them, **Crochet hyperbolic ball (Nash–Kuiper, C¹)**, is worth a special mention: it is a rare picture of a real theorem about regularity classes. Hilbert proved in 1901 that no *smooth* (`C²`) surface in ordinary space can carry the true hyperbolic metric on any patch that stays hyperbolic without bound — the curvature would have to blow up at the edge. Nash and Kuiper showed in the 1950s that dropping to `C¹` — one derivative, continuous, but no promise on the second — removes the obstruction completely: a `C¹` isometric copy exists on a disk of *any* size. The formula loaded by this preset is the explicit single-wave case of their construction, restricted to the radius `a = 1/√3` where the mismatch with the true metric is smallest; the one ripple you see is that mismatch, made visible. It is also, not by coincidence, what a real crochet hyperbolic surface looks like — a physical model of exactly this theorem, worked one stitch at a time. Selecting it also masks the domain to the disk the construction is valid on, through the feasible-set machinery covered next.

A second special example is **Mount Saint Elias (real terrain)** — not a formula at all, but the actual mountain: a digital elevation model of the 40 km around the summit (Was'eitushaa, Boundary Peak 186, 5 489 m, on the Alaska–Yukon frontier), from the United States and Canadian national elevation surveys, carried inside the program and exposed to the formula box as the function `elias(x, y)` — km east and north of the summit in, km of elevation out. Everything in the program works on it unchanged, because nothing in the program ever needed `f` to be a formula — only to be callable. The reason it is here is the frontier: the 1903 boundary is locally a straight line, it passes about six hundred metres from the summit, and the summit is on the Canadian side. Loading the example sets the feasible set to Alaska — one linear inequality, exactly a budget constraint — so the highest point of the United States is *not* the summit: press `0` and the optimiser pins it to the line, where the level curve is tangent to the frontier. Maximisation under a linear constraint, on a real mountain, with the Lagrange condition visible as geography. The surface wears its true climate — ice and snow with scattered rock, vegetation only near the sea — and the relief is real, so stretch the z-axis dial to exaggerate it the way an atlas would.

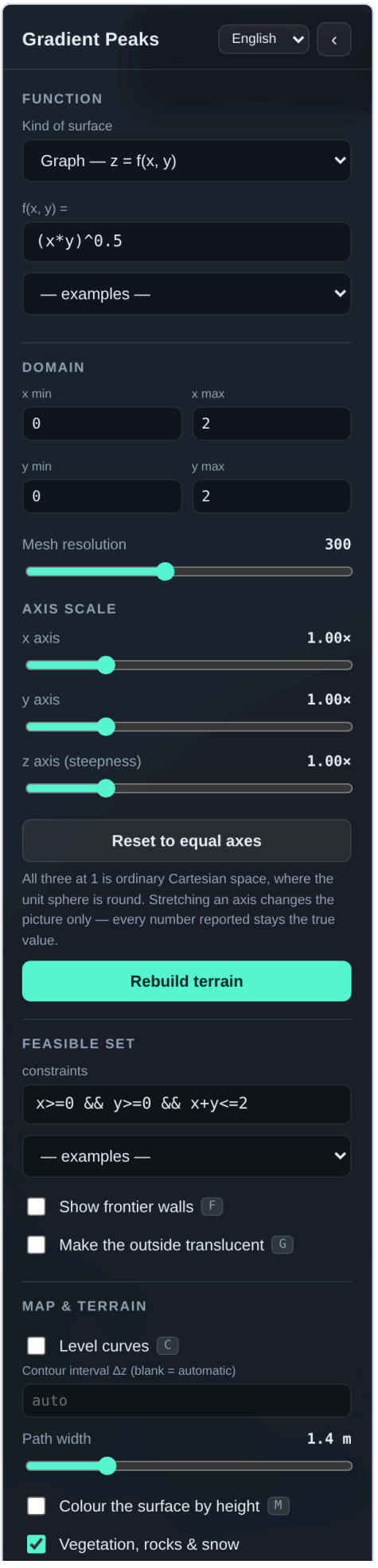
One refinement matters mathematically. What `elias(x, y)` evaluates is not the raw elevation grid but a **smooth approximation of it**: a two-dimensional Fourier (cosine) series fitted to the survey, kept within about 90 m of the data on average. A finite sum of cosines is infinitely differentiable — every derivative of every order exists at every point — so the gradient arrows, the tangent plane, the geodesics and the level curves all operate on an exact smooth object, not on interpolation artefacts. Interpolating every sample would also manufacture hundreds of relative maxima the size of a pixel; the fitted surface keeps only the massif's dozen real shoulders, so "find the relative maxima" remains a question with an honest answer. The apparent roughness — crags, snowfields, scattered rock and ice — is decoration standing *on* the surface. Mathematically, the mountain you walk is as smooth as any formula in the preset list, and the constrained maximum still sits exactly on the boundary line.

Mountains cut by a border

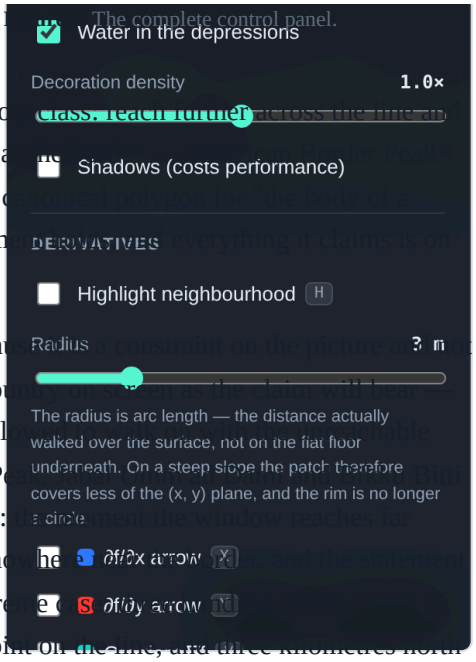
Under **Border mountains** in the examples menu are twelve real mountains that a frontier crosses below the summit. They come from an atlas of the rare configuration: a named mountain intersected by an international boundary whose *highest point* is nonetheless strictly inside one country. Choosing one loads the mountain, sets the domain to a window around the summit, and writes the frontier into the constraint box — as the feasible set for the country that does *not* own the peak. So the summit is out of reach by construction, and the best you can do lies somewhere on the line. Press `0` and the optimiser pins it there, at the point where a level curve runs tangent to the frontier. It is the Lagrange picture, and the constraint was negotiated by diplomats rather than invented for a problem set.

An aerial photograph appears in the top right corner with a short description of where you are: the two countries, which line the frontier is, and how far the summit stands from it. The thick red corridor drawn across the photograph is the atlas's own annotation — a schematic projection of the boundary, deliberately far wider than the real line so that it reads at that size. Where the atlas admits a picture is regional context rather than the mountain itself, the caption says *Regional view*.

Eight of the twelve are cut by **a line defined by a number**: the 49th parallel across the Rockies and Cascades, the 141st meridian between Alaska and the Yukon. In local coordinates those are exactly `y = constant` and `x = constant`, with no survey error to inherit — the constraint really is the legal line. The rest are straight treaty segments reproduced from their own defining endpoints: the 1848 Guadalupe Hidalgo line behind Otay Mountain and Tecate Peak, the sectors crossing Jabal Umm ad Dami and Bikku Bitti.



Each surface is a smooth Fourier fit to the public elevation model, exactly as Saint Elias is, and the window is the neighbourhood of the summit in which the claim is actually true. That last point is worth making to you eventually collect a second summit over there, whose top is nowhere nea has Canadian Border Peak less than three kilometres north of it. There is no mountain", so the window is chosen to be the widest one in which the statem screen to be checked.



The complete control panel.

How the frontier is placed in the frame is itself worth a minute in class, becau a choice of taste. The window is positioned to put as much of the *feasible* co ideally about two thirds of it, so a student sees mostly the ground they are all summit standing over the far third. Four of the twelve manage it — Tecate P at 60%, Kinnerly Peak at 55%. The rest cannot, and the reason is geography: enough across the line, it collects a second summit over there whose top is n the example exists to make becomes false. American Border Peak is the extr the 49th parallel the Canadian ground already stands 55 m above the best po there is a 2 398 m summit. The honest window there shows a few hundred metres of Canada and no more. Fraction and window size trade against each other exactly — the fraction times the width is the distance the window reaches across the line, and that distance is fixed by where the neighbouring peak stands — so a wider mountain to walk on is bought with a thinner strip of the far country, and the build chooses the strip. The builder tries the two-thirds band first, widest window first, and falls back through smaller fractions only when nothing in the band survives; what each mountain actually achieved is recorded with it. If you want more of the far country in frame than the example ships with, widen **y min** and **y max** yourself — and then watch the optimiser walk off the line, which is the lesson stated backwards and just as good.

One more presentation choice. The z axis opens **stretched**, three to nine times depending on the mountain, because at true scale two kilometres of relief across eighteen kilometres of ground is a swelling rather than a peak. It is a display scale only: `f`, its gradients, the level curves and every readout stay in real kilometres, and the z-axis dial puts it back to 1 whenever you want to show the class how gentle a real mountain is.

A real place: Universidad de los Andes, Santiago

The last entry in the examples list is not a landscape but a neighbourhood: the rectangle from 70°30′40″W to 70°29′15″W and from 33°24′S to 33°25′S, which is the ground around the Universidad de los Andes in San Carlos de Apoquindo, on the Andean foot slope at the eastern edge of Santiago. It is 2.19 km east–west by 1.84 km north–south, with 505 m of relief across it, and it opens at true vertical scale because a hillside that steep needs no help to read as one.

Three public sources go into it, and they are kept deliberately apart.

- **The ground is the function, and nothing else.** A finite cosine series fitted to the public elevation model, within about 5 m of it, and therefore infinitely differentiable — the same treatment Mount Saint Elias and the border peaks get, for the same reason.
- **The vegetation is surveyed, not inferred.** Everywhere else in this program what grows on a surface is deduced from height: this fraction of the relief is forest, that fraction is scree. Here ESA WorldCover records at 10 m whether each patch is tree cover, matorral scrub, dry grass, bare ground or built, and both the colour and the planting read that instead. The trees are where the trees are.

- **The buildings are real outlines.** 1 493 of them from Overture Maps, with real heights where the source states one — it states twelve — and a height inferred from the plan area where it does not.

None of the scenery is part of `f`, and that is the point of the example rather than a caveat about it. Walking into a lecture theatre does not change the altitude. A roof is not a local maximum. The gradient at a doorway is the slope of the hillside the doorway was cut into. A class that has spent a term maximising invented functions over invented budget sets can stand in a place they walk through every week and ask the same questions of it: which way is downhill from here, how steep is the walk up to the library, what does a level curve of altitude look like when it crosses a car park. Switch the level curves on and walk one.

Walking the frontier

Walk only along the frontier (`B`, in the Feasible set section) ropes the explorer to the constraint curve. Every step is projected back onto it, so pushing in any direction slides you along the line and pushing straight at it does nothing. Walk from one end to the other watching the height readout and you have solved the constrained problem on foot: the highest reading you can find is the number the optimiser prints. It works for a curved constraint as readily as a straight one — the curve is read out of whatever inequality is in the box, so a circle or a Cobb–Douglas budget set ropes you just as well.

Level curves only inside the feasible set (in Map & terrain) stops every contour exactly at the constraint. That is how an economics textbook draws indifference curves over a budget set — the curves live inside the feasible region, and the one that just touches the frontier is the answer.

If what you type does not make sense, a red message appears underneath naming the problem and the character where it went wrong. **The mountain you already had stays on screen** — a typing mistake never destroys your work.

Domain

x min, x max, y min, y max — the rectangle of the plane you are plotting over. The default is the square from 0 to 2 in both directions.

Mesh resolution — how finely the surface is divided into triangles. Higher is smoother but slower to build. 300 is the default; lower it to 100 or 80 on an old computer.

Axis scale — x axis, y axis, z axis — three dials, all at `1.00×` by default. That default is ordinary Cartesian space, where one unit of `x`, one unit of `y` and one unit of `z` are the same length and the unit sphere really is round. **Leave all three at 1.0 for teaching.** Stretching one changes the picture only: every number on screen stays the true, unstretched value, which is exactly the sort of mismatch that confuses students. The `z` dial exists for functions so flat you cannot otherwise see them, and the `x` and `y` dials for domains far longer than they are wide. **Reset to equal axes** puts all three back to 1.

Rebuild terrain — applies the function and domain. Pressing `Enter` in any of the boxes does the same thing.

Feasible set

constraints — the inequalities that define where the student is allowed to look for the maximum. Part 6 covers the notation.

Show frontier walls `F` — draws glowing walls along the border of the feasible region, standing on the surface. This is the "frontier of the country".

Make the outside translucent `G` — everything outside the feasible set fades to a ghost, so only the permitted region is solid. Turning both of these on is the clearest way to pose a constrained optimisation problem.

Map & terrain

Most of this section is live on all three kinds of surface — the terrain colouring, the world map, the vegetation and its density and size all come straight off the mesh, whatever shape it happens to be. Four items are graph-specific and stay hidden on a parametric or implicit surface: level curves and the contour interval need a height to be curves *of*, path width only means anything alongside them, and water is a plane at `z = 0`, which presumes a graph's particular idea of "down".

Level curves `C` — draws the contours on the surface as wide, walkable paths rather than hairlines, and *drapes them over the ground*: the centre of each path is at constant height, as a level curve must be, but the band itself follows the hillside it crosses instead of hanging in the air as a flat ring. Each path is coloured by its own height on the ten-colour ramp, so the colour states the altitude whether or not the surface itself is coloured.

Contour interval Δz — leave it blank and the program picks a round interval and shows you which one. Type a number and that is the interval, which is the point when the interval is the thing being taught. Ask for one so small that thousands of paths would be needed and it is widened, with a note saying so.

Path width — how wide those paths are, measured in explorer heights against the 1.80 m explorer, so they narrow along with the explorer rather than turning into a motorway once you shrink for the local-linearity demonstration. The default, two heights, is a footpath you can see from the air and still walk along — the same length a grid square's side is.

Colour the surface by height `M` — repaints the whole surface on the same ten-colour ramp the contours use, so the surface and its level curves agree — on every kind of surface, not only a graph. Turn it off and the surface goes back to the eight terrain bands — water, beach, deep vegetation, lighter vegetation, arid, volcanic rock, snow, cloud — which say the same thing in the language of a landscape. On a graph, combine either with `T` (look straight down) to turn the 3D scene into the flat map students already know.

The window follows the explorer — walk to an edge of the domain and the domain moves rather than stopping you. The plot is a window onto a function that has no edges, and a wall at `x = b` says something false about `f`. Reaching a side slides that axis along by `G` (or `H` for `y`) times its own width, in the direction you were walking:

- east, `x = b`: `[a, b]` becomes `[a + G(b−a), b + G(b−a)]`
- west, `x = a`: `[a, b]` becomes `[a − G(b−a), b − G(b−a)]`
- north, `y = d`: `[c, d]` becomes `[c + H(d−c), d + H(d−c)]`
- south, `y = c`: `[c, d]` becomes `[c − H(d−c), d − H(d−c)]`

Walking into a corner moves both axes at once. The width never changes — this is a pan, not a zoom — so the scale of the plot, and with it every reading taken against the 1.80 m explorer, survives the move. `G` and `H` default to 0.2, which buys a fifth of a domain width of walking before the next move; raise them for longer strides between rebuilds. The feature is left off in the consumer problem, where the domain is not a window onto anything but the set of bundles that exist.

Vegetation, rocks & snow — turns the scenery on and off, on any kind of surface. The scenery sits *on top of* the surface and never changes its shape; on a parametric or implicit surface it is scattered by triangle area rather than by parameter, so a sphere gets an even forest rather than a thicket at each pole.

Water in the depressions — graph only: a translucent lake surface at height zero, so any region where `f < 0` fills with water. There is no comparably natural "sea level" to cut an arbitrary closed surface with, so this stays hidden on the other two kinds.

Decoration density — how much scenery, on any kind of surface. Turn it down to speed things up.

Shadows — prettier, noticeably slower, and now shared by every kind of surface too. Off by default.

Paint the world map on it

Puts the Earth's map on whatever surface is showing: longitude across, latitude up, the plainest projection there is, with a graticule every 30° so the stretching is legible. Where the rectangle lands depends on what kind of surface it is — the parameter rectangle, or the domain of a graph, or longitude and latitude about the centre of an implicit surface.

Choose Parametric → Sphere and it becomes the globe it was traced from: the flat map's distortion visibly undoes itself, Greenland shrinks back to a large island, Antarctica stops being a continent-wide smear along the bottom. Then switch to a torus, or to a graph with a mountain on it, and watch the same rectangle distort in a way that belongs to *that* surface. No sheet of paper can carry a sphere without stretching it — that is Gauss's *Theorema Egregium* — and a class that has been told this can now see it happen in both directions.

The coastlines travel inside the program (Natural Earth, public domain, about 60 000 points), so this works with the network off, like everything else here.

Size of the trees

Scales the vegetation, rocks and grass from a tenth to five times. Left at 1× the scenery is sized against the world it stands on, the same rule a graph uses; a small sphere with a full-height forest on it is a picture worth seeing once and turning down afterwards.

Vegetation scales with the explorer — Lab only, and on by default: whatever the explorer scale dial does to the explorer, it now does to every tree, rock and blade of grass by exactly the same factor, so shrinking down for the tangent-plane demonstration shrinks the forest with you instead of leaving it suddenly towering. The size-of-the-trees dial still multiplies on top of that. Turn it off to hold the scenery at a fixed size while the explorer's own scale changes underneath it — useful for showing, deliberately, how large a small explorer would find an ordinary tree.

Grid on the surface

Its own section in the panel, and live on every kind of surface — a graph, a parametric patch and an implicit surface all have coordinates worth drawing, so unlike the level curves and the terrain colouring above it this one never greys out.

Coordinate grid on the surface `N` — draws the coordinate system *on* the surface, which is the mesh a textbook picture almost always has and which is not decoration: it is what lets you see what the coordinates are doing.

Every square is two explorers on a side, measured along the surface, and the same two in all three regimes — so the mesh is a ruler and not a texture, and a square means the same thing whichever kind of surface is on screen. How many squares across is this hill; how far apart are those two contours; how big is the neighbourhood the derivative is being read from. And because the square is measured in explorers, it shrinks when they do: take the size dial down a decade and the squares come down with you, and the surface you thought was curved turns out, at that scale, to be a plane. The panel says what one square is currently worth. Where two explorers to a square would need more lines than

a screen can carry — a 0.18 mm explorer on a 220 m plot — the spacing goes up in whole multiples, 1, 2, 5, 10, and the multiple is stated; a ruler with unreadable divisions is not more accurate, only unreadable.

What it draws depends on the kind of surface, and in each case it is the intersection of the surface with a family of level sets of the coordinates:

- **a graph** `z = f(x, y)` — the grid of the `(x, y)` plane lifted onto the surface: the curves `x = constant` and `y = constant` with `z = f`. Where the lines crowd together on screen, the surface is steep.
- **a parametric surface** `r(u, v)` — the parameter lines themselves. A sphere in polar coordinates has lines that crowd at the poles; a helicoid's do not meet at right angles; a Möbius strip's long parameter closes up while its short one does not. None of that is visible on a bare shaded surface, and all of it is worth seeing.
- **an implicit surface** `F(x, y, z) = 0` — where the surface cuts the coordinate planes `x = constant`, `y = constant` and `z = constant`. On a sphere these are the circles of latitude and longitude, found the same way the level curves on the terrain are.

The grid is drawn on *both sides*, so it stays readable when the explorer is walking on the inside of a torus — a single copy would be buried in the surface it is meant to be marking.

Build it from geodesics instead — offered on parametric and implicit surfaces, where the coordinates are not the surface's own. The other two grids inherit their spacing from something outside the surface: how somebody happened to write `r(u, v)`, or where the planes `x = constant` happen to fall. Neither is a property of the surface, and neither can honestly promise that a square is a square. A geodesic grid can, because it is built only from straightest paths and arc length:

1. take a perpendicular pair of directions in the tangent plane where the explorer is standing;
2. run the geodesic along one of them and mark it every L of arc length;
3. from each mark, run a geodesic in the direction perpendicular to the axis *there*;
4. do the same with the two directions swapped.

The result is the picture of geodesic normal coordinates. The two families cross at right angles along the axes and, on a curved surface, nowhere else — and that is not a defect in the drawing, it is the curvature. Geodesics that set off parallel do not stay parallel; how fast they converge or spread apart *is* the Gaussian curvature. On a sphere the squares close up as you go away from the explorer; on a saddle they splay open. It is the clearest picture of curvature a student is likely to meet, and it is anchored where the explorer was standing when it was drawn.

Derivatives

Highlight neighbourhood `H` — lights up a disc on the surface centred on the explorer. This is the neighbourhood in which all the rates of change are measured.

Radius — anywhere from half a metre to twelve, measured against the 1.80 m explorer, and measured **as arc length over the surface**: it is the distance actually walked from where the explorer stands, not the distance on the flat floor underneath. On level ground the two agree. On a slope of 1 you cover 2 m of ground in only 1.41 m of floor, so the patch shrinks in the `(x, y)` plane — and on a surface whose steepness varies with direction the rim stops being a circle at all. That is the honest local neighbourhood: the set of points two metres' walk away, exactly as the first fundamental form $g = (1 + f_x^2)\,dx^2 + 2\,f_x\,f_y\,dx\,dy + (1 + f_y^2)\,dy^2$ measures it. (Strictly it is the set of points reached by walking that far in a fixed compass direction, not the geodesic circle of that radius; the two agree to second order in the radius, and at these radii the difference is well below a pixel.)

🟦 **∂f/∂x arrow** `X` — a blue arrow drawn *along the surface* in the direction of increasing `x`.

🟥 **∂f/∂y arrow** `Y` — the same in red, for increasing `y`.

🟦 **Gradient ∇f** `V` — in teal and at double width, pointing in the steepest uphill direction.

Each one adds a badge to the top of the screen showing two numbers: the **instantaneous** derivative at the explorer's feet, and the **average** rate of change over the radius of the disc. Watching those two numbers converge as you shrink the radius is a lesson in itself.

🟨 **Directional derivative** `B` — freezes the explorer and lets you swing a direction vector **u** around the rim with the mouse. The badge shows $D_u f$ and the angle. To look around while in this mode, hold the **right** mouse button.

All four arrows are painted on the surface and reach the same arc length, so on a hillside they bend with the hillside; each ends in a flat arrowhead, proportioned to the width of its own shaft.

On a surface that is not a graph

A torus has no height function, so there is nothing to take a partial derivative *of* — and yet every object in this section still exists, because they are objects of the surface rather than of the formula. Pick Implicit or Parametric and the same five switches mean the following instead.

Geodesic circle around you `H` — the set of points you can reach by walking the given distance along a *straightest path*, in every direction. On a plane that is a circle of circumference 2πr. On anything curved it is not, and the badge reports the ratio C/2πr: below 1 the surface curves like a sphere, above 1 like a saddle. The shortfall is the Gaussian curvature — `C = 2πr(1 − K r²/6 + …)`, Bertrand and Puiseux, 1848 — and it is on screen and measured rather than asserted. On a graph you can switch the ordinary neighbourhood to this one too, with **measure it with geodesics instead**, and watch the two rims part company as the hill steepens.

First and second coordinate direction `X` `Y` — the blue and red arrows now point along the coordinate directions of *whichever grid is switched on*: the parameter lines `∂r/∂u` and `∂r/∂v`, or the directions in which the ambient `x` and `y` increase while staying on the surface, or the two axes of the geodesic grid. That is what makes them *partial*: the question has no answer until a coordinate system is named, and changing the grid changes the arrows. The badge reports the angle between them, which is 90° for longitude and latitude on a sphere, 90° by construction for the geodesic grid, and something else entirely on a helicoid.

Occasionally an arrow simply is not there, and the badge says the axis is perpendicular to the surface. Where the sphere `x²+y²+z²=1` meets the plane `x = 1` it is tangent to that plane: `x` is stationary, and "the direction in which x increases" is not a direction. Nothing is drawn, because there is nothing to draw.

Velocity vector `B` — the amber arrow is the direction you are actually travelling, tangent to the surface: γ'(0) for the path under your feet. On a graph the directional derivative is a number, the slope you feel; here there is no height to have a slope, and what is left is the vector that number was a derivative of. Walk, and it moves with you.

Tangent plane at your feet `P` — T_pS itself, drawn flat and straight in space, which is where those vectors live. The arrows curve away along the surface and the plane does not; they agree at your feet and part at second order, and shrinking the explorer closes the gap. That is differentiability, on a surface with no graph to draw it over.

The curve under your feet

Level curve you are standing on `J` — traces the contour through the explorer's exact height, not the nearest round one, and re-traces it continuously as they walk. It is coloured by that height on the ramp, and laid on the surface like the other paths.

Figure 3. The contour through the explorer's own height, and its tangent in magenta. Both lie on the surface: the paths bend over the hillsides they cross instead of hanging in the air.

Tangent line to that curve `K` — the tangent to that contour at the explorer's feet, in magenta, projected down onto the surface so that it hugs the ground rather than floating off it. Because a level curve keeps `f` constant, its tangent direction is perpendicular to `∇f`, and the projected line leaves the contour only at second order: the two run together near the explorer and part slowly further out. Shrink the explorer with the zoom dial and they close up again — which is what "tangent" means.

Tangent plane, and the explorer’s scale

Two sections in the panel now. The tangent plane is a statement about the graph of a function and greys out on the other two kinds of surface; the explorer’s scale is not, and stays live everywhere — an explorer on a torus has a size, and everything measured against them, the grid squares included, changes with it.

Tangent plane `P` — draws the flat plane that best matches the surface at the explorer's feet.

Explorer scale — changes the explorer's size by a factor of ten per notch. To the right they shrink: 1.8 m, then 18 cm, 1.8 cm, 1.8 mm, and finally 0.18 mm. To the left they grow, up to ten times life size at 18 m, which is the reading you want when the feature of the surface you are studying is bigger than a person. The camera, the walking speed and the measuring disc all change with them, so from the explorer's point of view nothing happens — but the patch of surface they occupy gets a hundred, a thousand, ten thousand times smaller. Activity 8 in Part 7 explains why this matters.

The mouse wheel does the same thing, the way it zooms a map, so you can change scale without going back to the panel. On a Mac trackpad, pinch.

Optimisation

Show optimum `O` — finds the highest point of the surface within the feasible set and marks it with a golden ring and a shaft of light you can see from anywhere. The report underneath gives the maximum value, where it is, whether it sits in the *interior* or on the *boundary*, and the size of the gradient there.

Teleport to the optimum — puts the explorer on that spot instantly.

View

Three ways of being in the scene.

- **First person** `1` — through the explorer's own eyes, walking on the surface. The slope is felt rather than seen.
- **Third person** `2` — over the explorer's shoulder. The best view for the derivative arrows, because you can see both the ground and the figure standing on it. On a sphere or a torus the camera rides behind the heading in the explorer's own frame, so the mouse steers them and the camera comes round with them, exactly as in first person; the vertical movement raises and lowers the camera instead. It also stays close: however far you pull back with the wheel, it will not put the surface between itself and the person it is following.
- **Drone** `3` — a flying camera, off the surface altogether.

Drone camera — visible only in drone mode, and offering the same choice again one level up: *first person* puts you in the aircraft, *third person* puts the camera behind it so you can see the drone itself.

Figure 4. The drone seen from its chase camera. The beam hanging beneath it is perpendicular to the plane `z = 0`, and the bright ring where it lands is the point of the domain the drone is over — here, still the spot the explorer took off from.

The drone flies level. `W` `A` `S` `D` move it over the `(x, y)` plane, `Space` and `Ctrl` set its altitude, and the mouse only aims the camera — so you can look straight down at the ground and still cross the domain at a constant height, which a camera that dives wherever it is pointed makes very hard. Hanging beneath it is a bright beam, perpendicular to the plane `z = 0`, ending in a bright ring on the surface: that ring is the point of the domain you are above, and watching it is how you keep track of where you are while flying.

Explorer's look — hiker, brick minifigure, blocky pixel figure or round buddy. All four are exactly 1.80 m tall, so the scale reference never changes.

Look straight down `T` flies the drone directly overhead, which projects the surface onto the plane `z = 0` — the flat map view. **Return to the centre** `R` rescues an explorer who has wandered off.

Direction indicator `I` — the small globe in the top right corner. In a first-person view of a surface with no landmarks — the inside of a torus, a Möbius strip, a paraboloid at one part in ten thousand — it is genuinely easy to lose track of which way you are facing, and losing it makes every number on the screen harder to read. The globe answers it: a cage of latitude and longitude with a small figure inside, turned the way the camera is turned, and the bearing and the elevation in figures underneath. In drone mode the figure becomes the aircraft, with its lens pointing where the camera points.

Hold `⌘` (`⊞` on Windows) or `Alt`, or press `Esc` and move the mouse, and it enlarges — those are the moments you are consulting it rather than glancing at it. The wires on the far side of the globe are drawn faintly, so it reads as a globe and not as a flat doily; when you are looking away from the reader you see the back of the figure's head.

The flattening sends the three axes to `x → (1, 0)`, `z → (0, 1)` and `y → (sin 0.9, cos 0.9)`, so forward runs up and to the right at about 38° above the horizontal and backwards runs down and to the left. It is an oblique projection rather than an orthographic one — the six cardinal directions land on the unit circle but the diagonals run past it — so the cage is an ellipse and not a circle. That is a real consequence of drawing the depth axis at an angle without foreshortening it, and it is what keeps forward from collapsing into the frame.

Border Run — the game

Border Run is not a separate program, and there is nothing to install or open: it lives inside the app you already have. Choose any of the border mountains from the examples list and, once the ground exists to stand on, a card appears in the lower middle of the screen offering the run. Press **A** (or click *Play it*) to take it, **B** for *Just explore* if you only wanted to look at the mountain — the sandbox is untouched either way — or **Y** for the full list of missions. From anywhere in the sandbox, **Start** on a controller opens that list too.

It is built for a controller: plug an Xbox-layout gamepad into a laptop, or pair one with a phone or tablet, and it plays without touching a keyboard. Everything also works from the keyboard, because one pad between four students is normal.

Being one program is the point of it. A student who has just walked the frontier and planted a flag presses **A** on the result card and is standing on the same mountain in the sandbox, with the answer now marked, the level curves one key away and — in the Lab — the flat heat map beside the scene showing exactly where the curve they were walking runs tangent to the line.

Every mission has the same shape, because it is the shape of the mathematics. You are dropped in the country that does *not* own the summit. The peak is in plain sight across the frontier and it is not yours. Somewhere along the line there is a highest point you are allowed to stand on — that is `argmax f` subject to `g ≤ 0` — and the job is to walk until you are standing on it and plant your flag. The score is how many metres you left on the table: within 10 m is three stars, 30 m two, 80 m one.

Crossing the border is allowed, and that is deliberate. A wall that physically stopped the player would quietly turn a constrained maximisation into an unconstrained one on a smaller domain. So a student may walk right up to the summit, stand on it, press A — and watch the game refuse the flag and name the country they are trespassing in. That refusal *is* the constraint, and feeling it is worth more than reading it.

The teaching moment is the one nobody plans: a student walks uphill, hits the frontier, and discovers that the only way left to gain height is to slide *along* it — and that they keep gaining until the ground stops rising in the direction the line runs. That is the tangency condition happening to them. The result card says so afterwards, in the mountain's own numbers.

CONTROL	DOES
Left stick / `WASD`	walk
Right stick / mouse	look
A / `Enter`	choose, confirm, plant the flag
B / `Esc`	back to the mountains
X	level curves on and off
Y	show the frontier
Start	clip the level curves to your own country
D-pad / `↑` `↓`	choose a mountain

Medals are remembered in the browser, so a class can come back to a half-finished set. The sandbox is still underneath — press `Tab` during a run to open the panel and show the formula the mountain is made of, or the constraint in the box.

5 Writing your own functions

Type into the `f(x, y) =` box and press `Enter`. The notation is the ordinary one you would use in a calculator or a spreadsheet, with two conveniences added.

Playing it with a gamepad

Any Xbox-layout controller drives the explorer directly — including on an iPhone or iPad, over Bluetooth, with nothing to install. Browsers expose gamepads through a standard interface, and Safari has done so since iOS 10.3, with full support for modern controllers since 14.5. The left stick walks and strafes; the right stick looks and turns; both work on a graph, on a sphere and on an implicit surface alike.

LEFT STICK	walk and strafe	**RIGHT STICK**	look, and turn the heading
RT	sprint	**LB / RB**	drone down / up
A	next view: first, third, drone	**B**	the neighbourhood
X	the coordinate grid	**Y**	the world map
D-PAD ↑ ↓	the explorer's scale	**D-PAD ← →**	camera zoom
START	show or hide the panel		

The View section shows which controller has been seen, by name. If it says there is none, press a button on the pad: for privacy, browsers keep a gamepad hidden until one is pressed, so a controller that is paired and working still reads as absent until you touch it. There is also a switch there to invert the right stick's up and down, which is remembered between sessions.

A stick held to the stop walks at exactly the speed holding `W` does, so nothing about the readings changes because you picked up a controller. The dead zone is measured as a distance from the centre rather than on each axis separately — which is what keeps a gentle diagonal push from registering as nothing — and the response is curved, so small movements near the centre are fine and the edge is still full speed.

One caveat about cheap telescopic controllers, including some Machenike models: many have a mode switch, and in "keyboard" mode they send letters rather than presenting themselves as a gamepad. In that mode the line in the View section will keep saying there is no gamepad — but the pad will very likely work anyway, because it is sending `W` `A` `S` `D` and the arrow keys, which this program already answers to. If one mode does nothing, try the other.

How fast the explorer walks

Walking speed in the View section multiplies the ordinary pace, from about a quarter to twenty-five times. The pace itself is already measured in explorer heights per second, so an explorer a tenth the size still crosses the same ground in the same time; this is for when you want to get to the far side of a torus without narrating the journey. Holding `Shift` still sprints on top of it.

The rules

SYMBOL	MEANING	EXAMPLE
`+ - * /`	add, subtract, multiply, divide	`x/2 - y`
`^`	raise to a power	`x^2`, `x^0.5`, `x^-1`
`( )`	grouping	`(x+y)^2`
`| |`	absolute value	`|x-y|`
`pi`, `e`	the usual constants	`sin(pi*x)`

> Two conveniences
>
> **You may leave out the multiplication sign.** `2x` means `2*x`, `x y` means `x*y`, and `3(x+y)` means `3*(x+y)`.
>
> **Powers bind tightest.** `-x^2` is `-(x^2)`, not `(-x)^2` — the same convention as in your lecture notes.

Functions you can use

`sin` `cos` `tan` `asin` `acos` `atan` `atan2` `sinh` `cosh` `tanh` `exp` `ln` `log` `log2` `log10` `sqrt` `cbrt` `abs` `sign` `floor` `ceil` `round` `min` `max` `pow` `hypot` `mod` `clamp` `step` `gauss`

`hypot(a,b)` is `sqrt(a^2+b^2)`. `gauss(t)` is the bell curve `e^(-t²/2)`; `gauss(t, centre, width)` shifts and stretches it.

Examples that work

TYPE THIS	YOU GET
`(x*y)^0.5`	The Cobb–Douglas surface. Undefined unless `x` and `y` are both positive or both negative.
`x^0.3*y^0.7`	Cobb–Douglas with unequal shares.
`1-x^2-y^2`	A dome. One clean interior maximum at the origin.
`x^2-y^2`	A saddle. Useful for showing a point where the gradient vanishes but there is no maximum.
`sin(3x)*cos(3y)/2`	An egg carton: many local maxima.
`0.6*sin(2x)+0.4*cos(3y)+0.3*x*y`	Rolling hills with lakes. The prettiest one to walk in.
`-(x^2+y^2)^0.5+1`	A cone. The tip is a maximum where the function is *not* differentiable — see Activity 8.
`x*y*exp(-(x^2+y^2))`	Four alternating peaks and troughs.

Mistakes, and what the program tells you

YOU TYPE	YOU SEE	THE FIX
`sin(x`	Missing ")" after sin((at character 6)	Close the bracket.
`x*yy`	Unknown name "yy" (at character 3)	Only `x` and `y` are variables.
`2+`	Unexpected end of expression	Something is missing after the `+`.
`sin`	"sin" is a function — write sin(...)	Give it an argument.

> Where a function is undefined, there is simply no ground
>
> `(x*y)^0.5` has no real value when `x` and `y` have opposite signs. Plot it over a domain that crosses the axes and two quadrants come out **empty** — and the explorer will not walk into them. A note appears at the bottom of the screen telling you what fraction of the domain is undefined. This is not a bug to be worked around; it is a genuinely useful picture of a function's domain.

6 Writing feasible sets

The **constraints** box takes inequalities in `x` and `y`. Whatever satisfies them is the feasible set — the region in which the maximum must be found.

SYMBOL	MEANING
`<=` `>=`	less than or equal, greater than or equal
`<` `>`	strictly less, strictly greater
`&&`	**and** — both conditions must hold
`||`	**or** — either will do
`!`	**not**

You may chain comparisons the way you would write them by hand: `0 <= x <= 1` means exactly what it looks like. Leaving the box empty means "no constraint at all".

Examples

TYPE THIS	YOU GET
`x>=0 && y>=0 && x+y<=2`	A triangle — the classic budget set.
`x^2+y^2<=1`	A disc of radius 1.
`x+y<=2 && 2x+y<=3 && x>=0 && y>=0`	Two budget lines at once: a four-sided region.
`x*y>=0.3 && x+y<=2.5`	A curved constraint against a straight one.
`0<=x<=1.5 && 0<=y<=1.5`	A rectangle, written with chained comparisons.

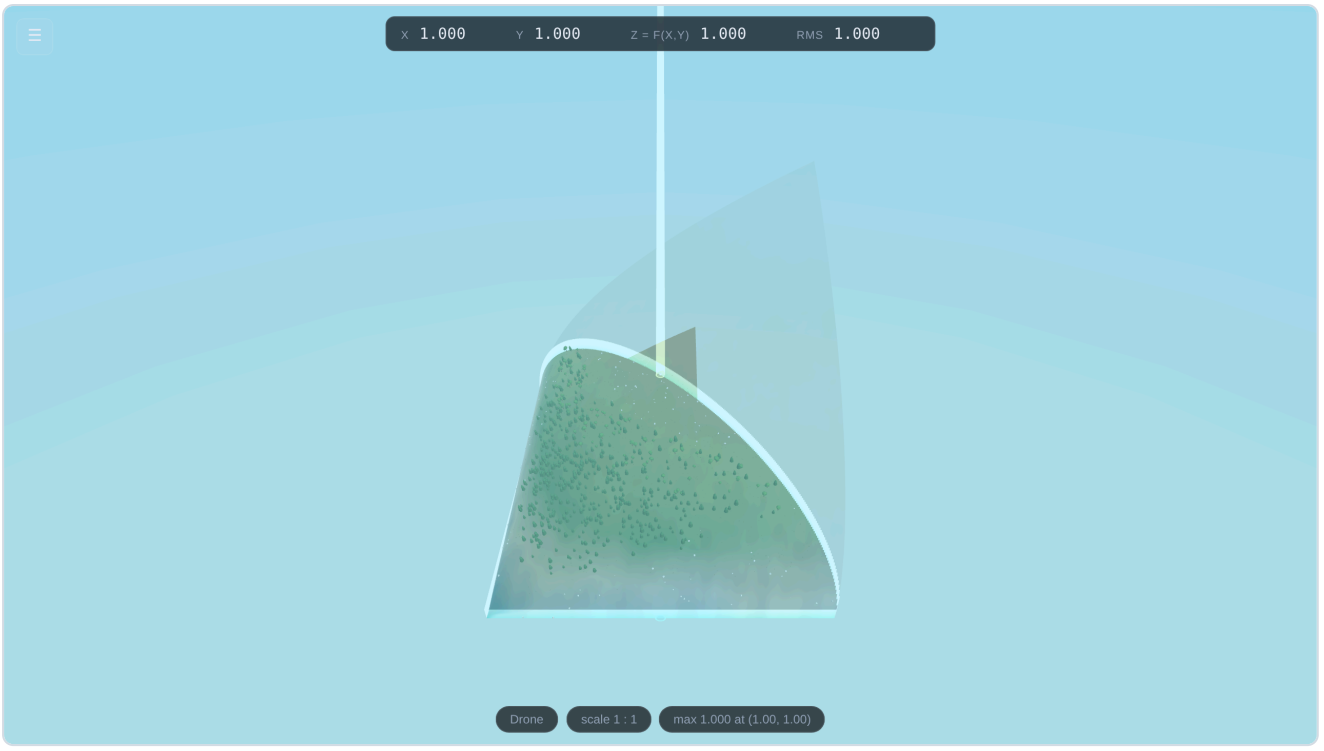


Figure 5. `x>=0 && y>=0 && x+y<=2` with both feasible-set switches on. The permitted triangle is solid ground with its scenery; everything outside has faded to a ghost. The shaft of light marks the constrained maximum.

7 Ten classroom activities

One for each feature. Each takes five to ten minutes and ends with something the students can state in words.

Activity 1 — A function is a landscape

Set up: the program as it opens.

Do: Look at the surface from the drone. Press `2` and walk uphill. Ask the class to call out what `z` is doing as you climb, and then to say what `z` *is*.

The point: the height of the ground under your feet is the value of the function at the point you are standing on. Everything else in the course is a statement about this landscape.

Activity 2 — Smooth surfaces, rough scenery

Do: From the air, note that the mountain looks perfectly smooth. Now turn **Vegetation, rocks & snow** off and on.

The point: The trees and boulders are placed *on* the surface; they never change its shape. A differentiable function has no corners no matter how close you look — but real mountains are covered in things that do. Being able to separate the two is worth a minute of discussion.

Activity 3 — Where you are allowed to stand

Set up: constraints `x>=0 && y>=0 && x+y<=2`. Turn on **Show frontier walls** and **Make the outside translucent**.

Do: Walk to the wall and try to see over it. Ask: where on this triangle is the ground highest?

The point: A constrained optimisation problem is a question about a region, not about a formula. Students who can see the region rarely get the geometry wrong afterwards.

Activity 4 — The same surface from three heights

Do: Switch between `1`, `2` and `3`, then press `T` to look straight down.

The point: The top-down view is the flat picture in the textbook. Showing students that it is the *same object* seen from above dissolves a surprising amount of confusion about contour diagrams.

Activity 5 — Level curves are paths you can walk

Set up: turn on **Level curves** `C` and **Topographic colours** `M`.

Do: Stand on a contour line and walk *along* it, watching `z` in the readout. Then walk across the lines and watch it change quickly.

The point: A level curve is where the function does not change. Walking one and watching the number hold still makes that concrete. Press `T` afterwards and the same lines become an ordinary contour map.

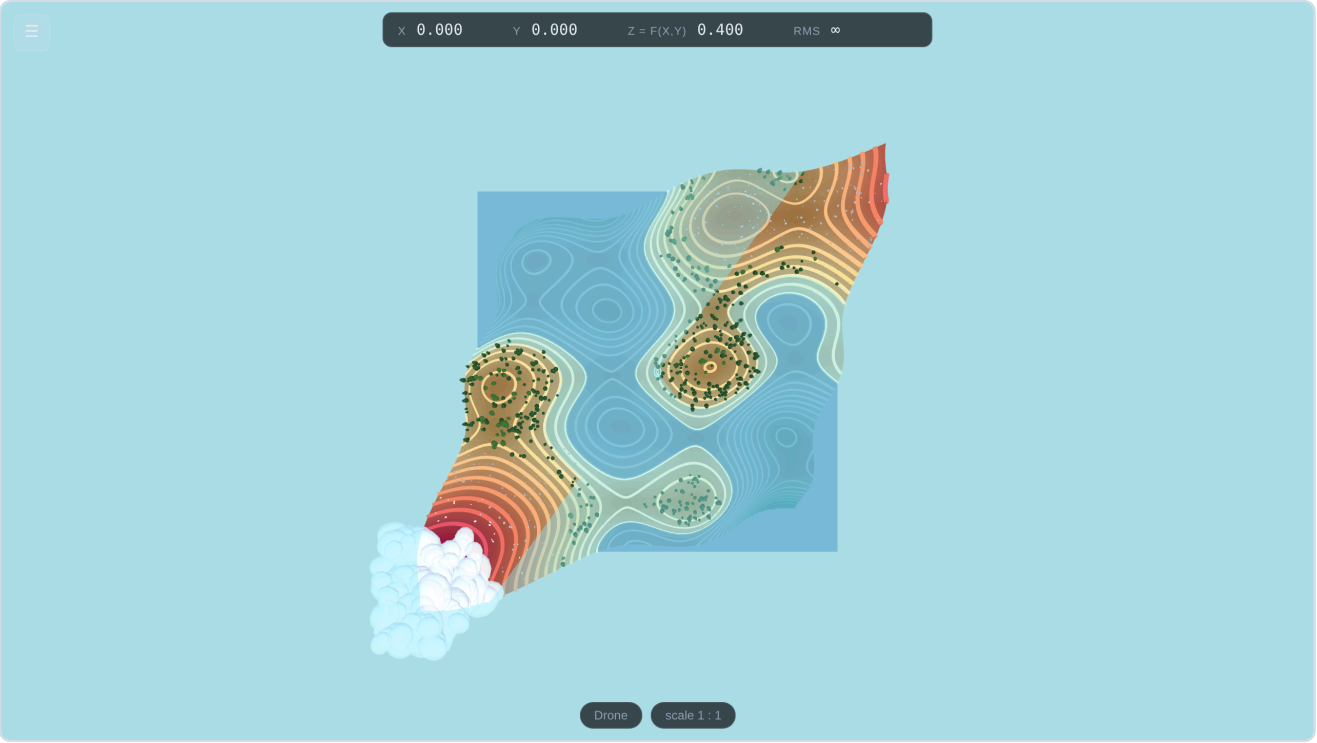


Figure 6. The same landscape from directly overhead with contour lines and map colours on. This is the picture in the textbook — and the students have just been standing in it.

Activity 6 — Reading the partial derivatives off the ground

Set up: `f(x,y) = (x*y)^0.5`. Turn on **∂f/∂x** `X`, **∂f/∂y** `Y` and **Gradient** `V`. Walk to roughly `(1, 1)`.

Do: Read the three badges. At `(1,1)` they say `∂f/∂x = 0.500`, `∂f/∂y = 0.500`, `‖∇f‖ = 0.707`. Check by hand: the partial derivatives of `√(xy)` at `(1,1)` are both `½`, and `√(¼+¼) = 0.707`.

Then: point out that the teal gradient arrow sits exactly between the blue and red ones, and that it is the steepest way up. Walk to a different point and watch all three swing round.

The point: the gradient is not an algebraic accident, it is a direction you can see and walk in.

Figure 7. The three arrows, drawn following the curve of the ground rather than as straight lines through the air. Blue is ∂f/∂x, red is ∂f/∂y, and the double-width teal arrow is the gradient. Here the red arrow has almost no length, because ∂f/∂y is nearly zero — so the gradient points along the blue one.

Activity 7 — Every other direction

Set up: turn on **Directional derivative** `B`.

Do: The explorer freezes. Swing the mouse slowly and watch $D_u f$ in the badge. Find the direction where it is largest — it lands on the gradient. Find where it is zero.

The point: The direction of zero directional derivative is the level curve, at right angles to the gradient. Students usually learn that as a formula; here they find it by hand in about fifteen seconds.

Activity 8 — Why differentiable means "flat if you look closely"

Set up: turn on **Tangent plane** `P`. Leave the zoom ruler at 1:1.

Do: Note that the yellow plane matches the surface underfoot but peels away from it a few metres out. Now drag the **Zoom-in ruler** one notch at a time and watch what happens.

The point: by 1:1000 the surface and its tangent plane are indistinguishable. That *is* differentiability: not a formula, but the fact that a small enough piece of the surface is a plane. Follow it with the cone `-(x^2+y^2)^0.5+1` and zoom in on the tip — it stays a point however far you go. That is what a non-differentiable point looks like.

Figure 8a. At 1:1 the tangent plane (yellow) clearly leaves the surface.

Figure 8b. The same spot with the explorer shrunk to 1.8 mm. The surface has become its tangent plane.

Activity 9 — The maximum, and why the gradient is not zero there

Set up: `f(x,y) = (x*y)^0.5`, constraints `x>=0 && y>=0 && x+y<=2`, **Show frontier walls** on, **Show optimum** `O` on.

Do: Read the report. It says the maximum is `1.0000` at `(1.0000, 1.0000)`, that it lies **on the boundary of the feasible set**, and that `||∇f||` there is `0.7071` — *not* zero. Teleport there, turn on the gradient arrow, and see it pointing straight at the wall.

Then: switch the feasible set off and press `O` again. The maximum jumps to the far corner of the domain.

The point — and it is the reason this program exists: at an unconstrained maximum the ground goes flat and the gradient dies. At a *constrained* maximum it need not: the ground is still climbing, and what stops you is the border. Students who have stood at that spot and seen the gradient arrow jammed against the wall do not later write "set the gradient to zero" on a constrained problem.

Activity 10 — The marginal rate of substitution, read off the ground

Set up: `f(x,y) = (x*y)^0.5` over `[0,2]²`, **Level curve you are standing on** `J` on, its **tangent line** `K` on, third person.

Do: Watch the `RMS` figure at the top while you walk along the contour you are standing on. It does not move: a level curve of $f = (xy)^{1/2}$ is a rectangular hyperbola `xy = c`, and on it `RMS = |y/x|`, which is the slope of the ray you are walking around, not of the path under your feet. Now walk *across* the contours instead, straight up the gradient, and it still barely moves — for this function the rate depends only on the ratio `y/x`.

Then: go to the point `(1, 1)`. `RMS = 1`: one unit of `x` substitutes for exactly one unit of `y`. Walk to where `x` is four times `y` and it reads `0.25`. Ask the class to predict the reading before each move.

The point: the marginal rate of substitution is not an extra formula bolted on to the derivatives — it is the slope of the indifference curve, and the indifference curve is a path the student has just walked along. At the constrained optimum of Activity 9, that number equals the price ratio, and both are visible in the same frame: the tangent to the level curve and the frontier wall lie flat against each other.

8 Surfaces that are not graphs

Everything so far has been a *graph*: one height `z` above every point `(x, y)`. That is a strong restriction, and most of the surfaces a geometry course cares about break it. A sphere has two heights above the same point. A torus has four. A Möbius strip has no consistent notion of "up" at all.

The **Kind of surface** menu at the top of the panel offers two other ways of writing one down.

Implicit — F(x, y, z) = 0

Every point of space where one equation is satisfied. `x^2+y^2+z^2-1` is the unit sphere; `(sqrt(x^2+y^2)-1)^2+z^2-0.16` is a torus. There is no height function, so no level curves and no `∂f/∂x` — but there *is* a normal, namely `∇F`, and where the normal exists you can stand.

Parametric — r(u, v)

A sheet swept out by two parameters. The **Named surface** menu fills in nine of them with dials for their own parameters — sphere, torus, pseudosphere, hyperboloid of one sheet, catenoid, helicoid, Möbius strip, Klein bottle and cross-cap. Choosing one writes its formulas into the same `X(u,v)`, `Y(u,v)`, `Z(u,v)` boxes you could have typed them into, so a named surface is a worked example rather than a hidden mode. Edit them afterwards and see what happens.

Standing on one

Click the surface. That is the whole interface. An implicit surface has no natural starting point, and a parametric one's parameter origin is an artefact of how somebody wrote it down, so the honest thing is to let you choose. The explorer lands exactly where you clicked and the view drops to third person.

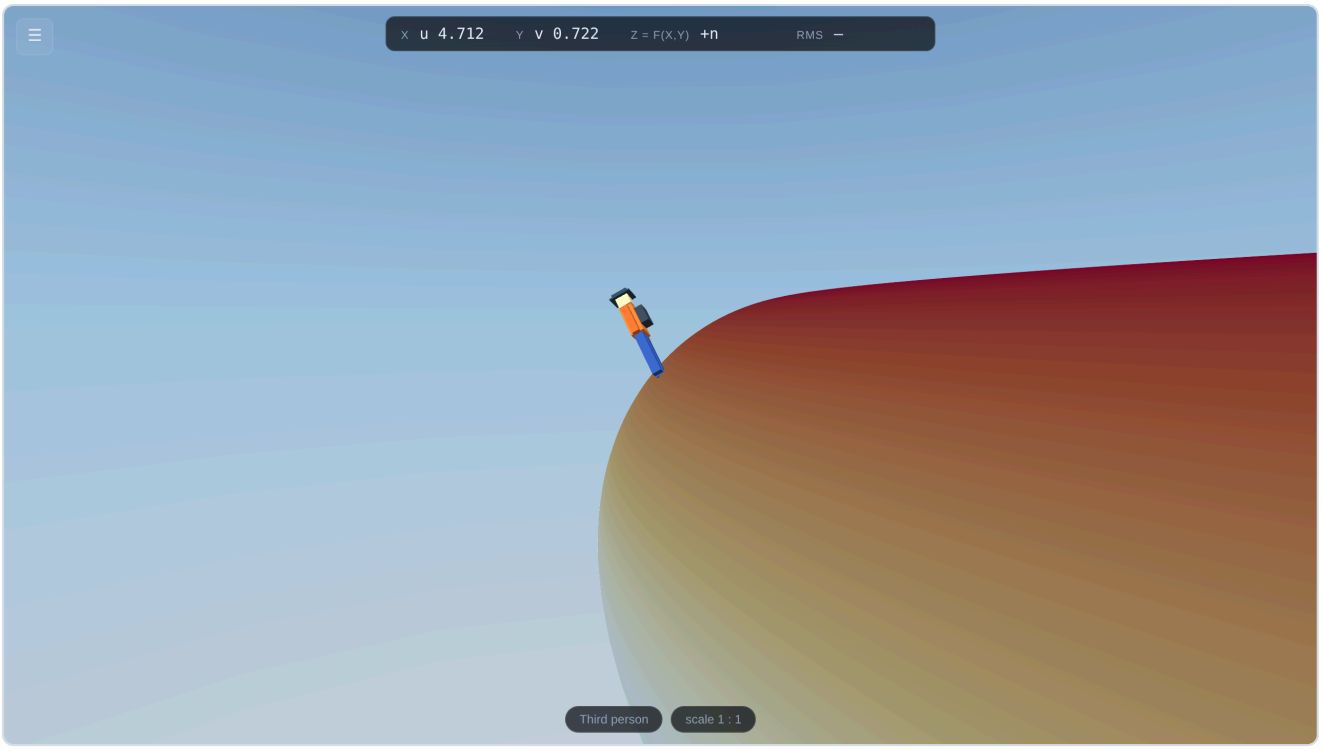


Figure 9. Standing on a torus. There is no height function here and no contours, but there is a normal at every point, and the explorer stands along it.

Then `W` `S` walk, `A` `D` step sideways, the mouse turns, and the wheel pulls the camera back. `1` is the explorer's own eyes, `2` the camera watching them, `3` the drone.

Up is the normal to the tangent plane. That single choice is what makes a sphere feel like a planet rather than a ball you are sliding around on: the horizon tilts with the ground, because on a sphere it does. **Which way is up** can pin it to a world axis instead, for a student who finds a tumbling horizon harder to read than a fixed one.

> Walking is done properly, and that costs something worth knowing
>
> A step of a given length in the parameters is *not* a step of a given length on the surface — near the pole of a sphere, a step in longitude covers almost no ground. Moving at a constant speed therefore means solving the first fundamental form $[E\ F;\ F\ G](du,dv)^{\mathsf{T}} = (r_u \cdot w,\ r_v \cdot w)^{\mathsf{T}}$ at every step. On an implicit surface there is no chart at all: the step is taken in the tangent plane and then pulled back onto $F = 0$ by Newton's method along ∇F. Neither is expensive, and both are why the explorer stays on the surface instead of drifting off it.

Walking straight ahead

On a graph you walk in the plane and the surface goes up and down underneath you. On a torus there is no plane to walk in, and "straight ahead" needs a definition. The one used here is the only one that does not depend on how somebody wrote the surface down: **the explorer follows the geodesic** through where they are standing, with their velocity as its initial condition — the straightest path the surface allows. On a sphere that is a great circle. On a plane it is a straight line. On a torus it is one of a family that mostly never closes up, and watching one wander round and round without ever repeating is worth a lecture on its own.

Turning is a rotation of the velocity inside the tangent plane; walking is the geodesic that velocity starts. Between the two, the heading is *parallel-transported* — carried along with no rotation about the surface's normal, which is what carrying a direction along a path has to mean when there is no fixed direction to compare it with. Walk a great circle all the way round and the heading comes back exactly as it set out. Walk a closed loop that is *not* a geodesic — round

a circle of latitude, say — and it does not, and the angle it comes back turned by is the holonomy: the area enclosed, times the curvature. That is the Gauss–Bonnet theorem, and the explorer can walk it.

Inside, outside, and one-sided surfaces

Walk on the inside flips which side of the surface the explorer stands on. On a sphere or a torus that is a real choice: the surface has two sides, and you can be on either. The note under the menu says which kind of surface you have picked.

On a **Möbius strip** the toggle is greyed out, and the reason is the lesson. The strip has only one side. Walk one full lap and the explorer comes back *upside down*, standing on what you would have called the other face, without ever having crossed an edge. That is what non-orientable means, and it is much harder to disbelieve when you have just walked it.

The **Klein bottle** and the **cross-cap** carry a second warning: they are three-dimensional *immersions*. The surface passes through itself, because the true embedding needs a fourth dimension to get out of the way. Walking through the intersection is the price of drawing it in a room.

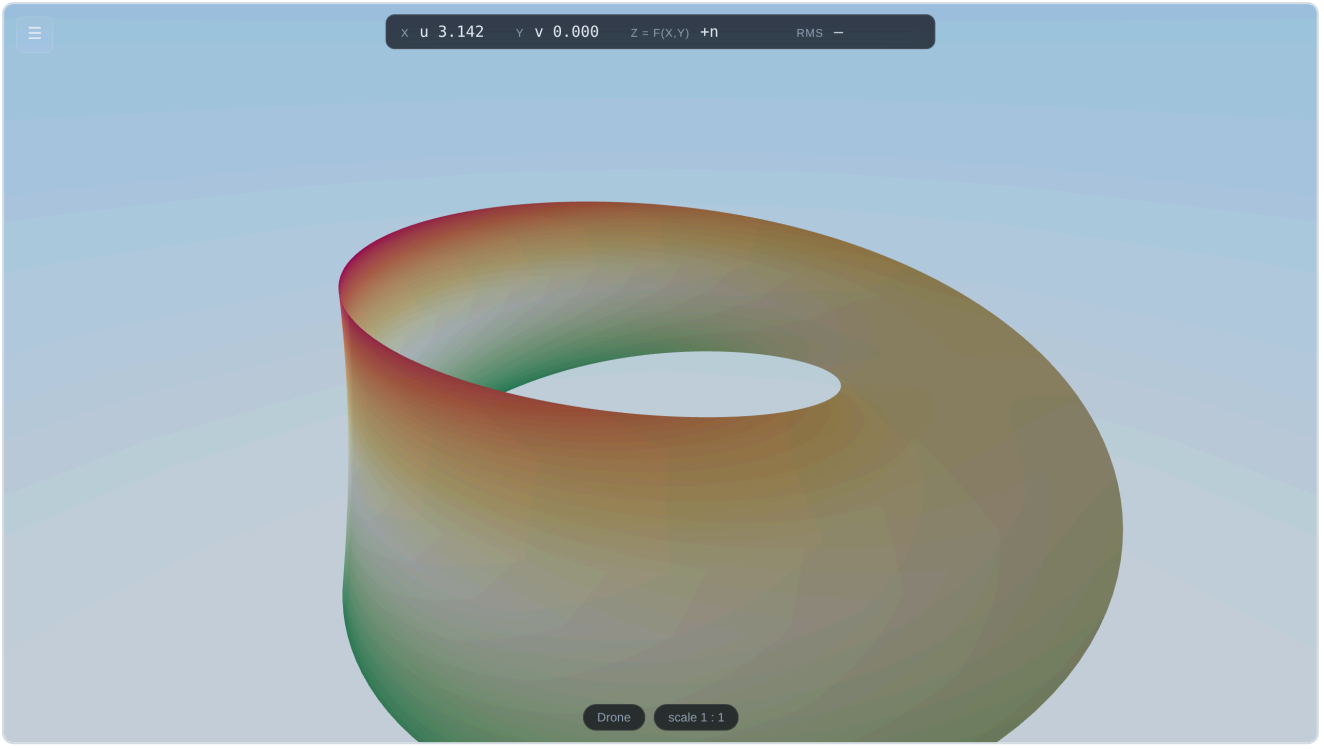


Figure 10. A Möbius strip. The inside / outside toggle is greyed out because there is no inside: one lap of this band brings the explorer back upside down.

Activity — one lap of a Möbius strip

Set up: Parametric → Möbius strip. Click the middle of the band. Third person.

Do: Note which way the explorer's head points relative to the camera. Hold `W` and walk all the way around the band, watching the readout — the `u` figure climbs to 2π and wraps back to zero. Look at the explorer.

The point: nothing was flipped, nothing crossed an edge, and the explorer is upside down. "This surface has one side" stops being a phrase to memorise.

The strip itself keeps score for you. Its colour is one lap's worth of journey: pure white where the explorer starts, deepening continuously to its darkest blue at the far side of the lap, and fading back to pure white on return. And at the starting point stands a golf flag whose pole runs straight through the band — a **blue** pennant on the side the journey begins, a **white** one out the other face. Walk the lap: the ground under your feet returns to exactly the same white, at exactly the same point — but the flag you are now facing is the white one. The colouring, like any continuous marking on the surface, cannot tell the two visits apart; only the flag, which is not part of the surface, can. That is non-orientability in one picture. (The *Height colours* toggle still repaints the strip as an ordinary height ramp if you want it.)

9 The consumer problem

Tick **Consumer problem** at the top of the panel and the program starts speaking economics. Nothing about the mathematics changes — that is exactly the point being made — but the words on screen become the ones a microeconomics course uses.

IN GENERAL	IN THE CONSUMER PROBLEM
the function `f(x, y)`	the utility function `u(x, y)`
the feasible set	the budget set
level curves	indifference curves
`RMS` in the readout	`MRS`, the marginal rate of substitution
the constrained maximum	the consumer's optimal bundle

The dials

Utility function — Cobb–Douglas, CES, perfect substitutes, or quasi-linear. **Share a** is the exponent, the weight, or the elasticity parameter, depending which you chose; the label changes to say which. **price of x**, **price of y** and **income** are what they say.

Changing any of them rewrites two boxes you can see: the utility goes into `f(x, y) =` and the budget set into the constraints box as `x>=0 && y>=0 && p_x*x+p_y*y<=m`. The domain is reframed on the budget set so the frontier is visible rather than jammed against the edge of the world. Nothing is hidden: a student can read the formula the dials produced, change it by hand, and see that "the consumer problem" is one particular `f` over one particular feasible set.

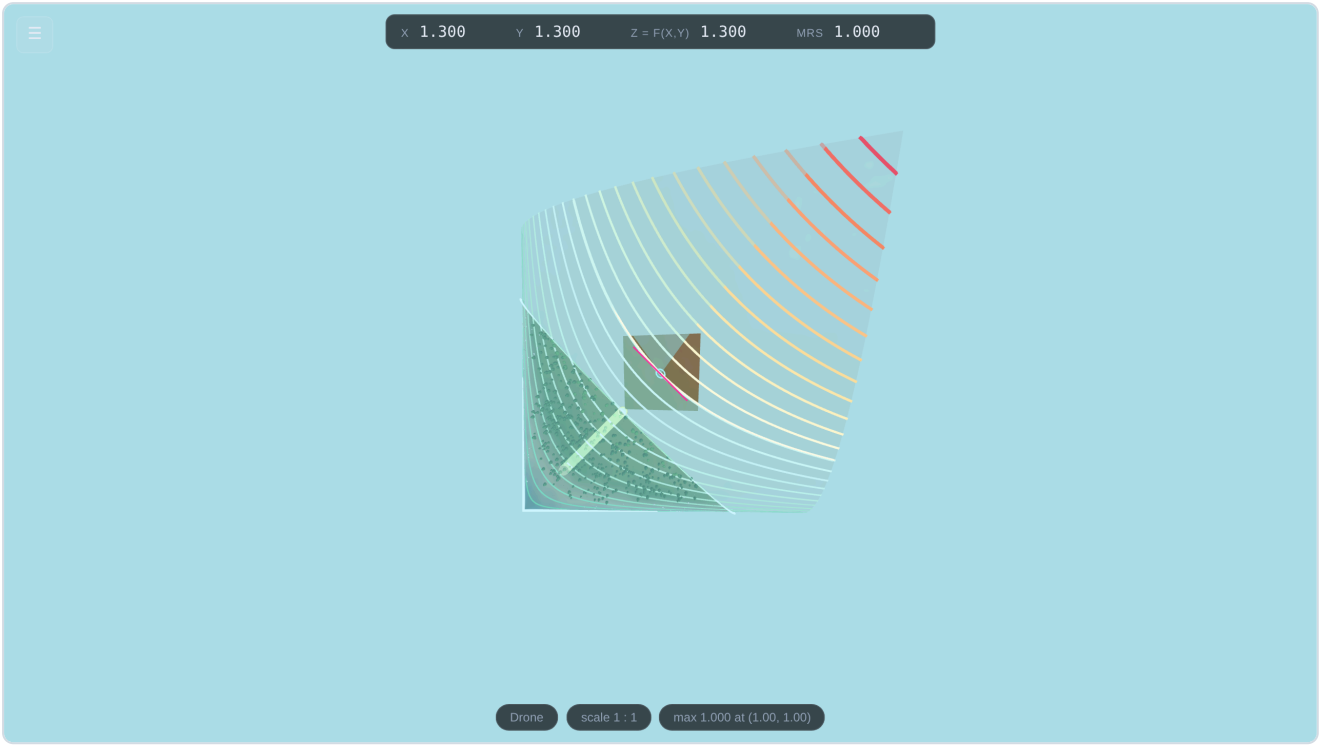


Figure 11. The consumer problem from directly overhead: indifference curves, the budget set, and the optimal bundle where one curve touches the budget line.

Activity — tangency, walked

Set up: Consumer problem on, Cobb–Douglas, `a = 0.5`, `$p_x = p_y = 1$`, `m = 2`. Turn on **Show optimum**, **Indifference curve you are standing on** and its **tangent line**.

Do: Read the report: the maximum is `1.0000` at `(1.0000, 1.0000)`, on the boundary. Teleport there. Read `MRS`: it is `1.000`, and so is `$p_x/p_y$`. Look at the tangent to the indifference curve and at the budget wall: they lie flat against each other.

Then: set `$p_x = 3$`. The optimum moves to `(0.333, 1.000)` — the textbook `$m/2p_x$`, `$m/2p_y$` — and at that bundle `MRS` is `3`, the new price ratio. Ask the class to predict the next one before you turn the dial.

The point: "MRS equals the price ratio at the optimum" is usually a line of algebra. Here it is a picture of two lines lying on top of each other, at a spot the student walked to.

10 The Lab — the flat map beside the scene

There is a second version of the program, `lab.html`, reached from the link at the bottom of the Help section (and back again from the same place). It is the same program, with the same controls doing the same things. Two differences.

The controls get out of the way. The rail is narrower and quieter, the `Tab` key hides it entirely, and the ⛶ button beside the language toggle goes full screen so the browser's own tabs and address bar disappear too. What is left is the surface.

A panel in the corner shows the picture your textbook draws. The domain seen from above, with the same level curves in the same colours, the same tangent line in the same magenta, and the explorer as an exaggerated dot. Everything on it is the same state as the scene behind it, so walking in the 3D view walks the dot on the map.

Figure 12. The Lab. The panel in the corner is the same domain seen from above — the same contours in the same colours, the same tangent, and the explorer as a dot. Both pictures are driven by one state.

What the panel can show

Heat map on the level-curve ramp — the default. The background is coloured by the same ten-colour ramp as the contours and the surface, so a colour means the same height in all three places.

Classical heat map — blue through green to red, the palette a student will have met on every other heat map. Deliberately different, so the two cannot be confused.

Looking straight down at the surface — not a drawing at all, but the scene itself rendered from directly overhead into the same rectangle. Trees, water, frontier walls and the explorer, seen as a map. This is the honest version of "project the surface onto `z = 0`"; the drawn modes are the idealised one.

Nothing — hide the panel.

Panel opacity and **Panel size** do what they say. The default is a third of the window's height by a fifth of its width. The panel never intercepts a click: it is a readout, not a control, and the mouse goes straight through it to the scene.

Activity — the same curve, twice

Set up: the Lab, any function with real relief, level curves on, the curve under your feet on, third person.

Do: Walk along the contour under the explorer's feet, in the 3D view, staying at the same height. Watch the dot on the panel: it traces a closed loop, and `z` in the readout does not move.

Then: walk straight up the gradient instead, crossing contour after contour. On the panel the dot cuts across the rings.

The point: the flat picture in the textbook and the hillside under your feet are the same object. Most students are told this. Here they can watch it.

11 Giving it to your students

Four ways, easiest first. Option A needs nothing at all; option D gives you a permanent web address. All four are free.

Option A — Send them the file

Email `Gradient-Peaks.html` as an attachment, or drop it in your course folder, or put it on a USB stick. Students double-click it. That is the whole procedure.

> One caution about email
> Some mail systems strip or block `.html` attachments. If it does not arrive, put the file in a shared drive (OneDrive, Google Drive, Dropbox) and send the link instead — but tell students to **download** it and then open it, not to preview it in the browser, since some drive previewers will show the file's text rather than run it.

Option B — Upload it to your course page

Moodle, Canvas, Blackboard and their relatives all accept a file upload. Add `Gradient-Peaks.html` as a resource. Students click it and it runs.

Option C — Netlify Drop: a real web address in about two minutes

This gives you a link you can put on a slide. No account is needed to start.

1. Go to `app.netlify.com/drop` in your browser.
2. Drag the `website` folder — the whole folder, not the files inside it — onto the dashed box on that page.
3. Wait a few seconds. You are given an address like `https://cheerful-lion-12ab34.netlify.app`.
4. Open it to check, then share it. Anyone with the link can use it, on any device.

Create a free account within 24 hours if you want to keep the address permanently or give it a friendlier name.

Option D — GitHub Pages: permanent, free, and version-controlled

More steps, but the address never expires and updating it later is easy. If the project already lives in a GitHub repository:

1. Open the repository on `github.com`.
2. Click **Settings** (the tab along the top of the repository, not your account settings).
3. In the left sidebar, click **Pages**.
4. Under **Source**, choose **GitHub Actions**.
5. The workflow already included in the project (`.github/workflows/pages.yml`) publishes the `app/` folder automatically on every push to the main branch.
6. After a minute or two the same Pages screen shows your address, of the form `https://<your-username>.github.io/<repository>/`.

> Which folder do I upload?

> For options C and D you publish the **`website/`** folder, because a real web server can serve its separate files correctly. Option A and B use the single `Gradient-Peaks.html` instead. Both contain the same program.

12 Installing it as an app

Once the program is at a web address (options C or D above), it can be installed like an app — its own icon, its own window, no browser bar, and it works offline afterwards. There is no app store involved, no payment, and no developer account.

DEVICE	HOW
iPhone / iPad	Open the address in **Safari** (this does not work in Chrome on iOS). Tap the **Share** button — the square with an arrow — then scroll down and tap **Add to Home Screen**.
Android	Open the address in **Chrome**. Tap the three-dot menu, then **Install app** or **Add to Home screen**.
Windows	Open in **Chrome** or **Edge**. Click the small install icon at the right-hand end of the address bar (a monitor with an arrow), then **Install**.
macOS	In Chrome or Edge, the same install icon. In **Safari**, choose **File → Add to Dock**.

After installing, the program keeps a copy of itself on the device. Students can use it on the bus with no signal.

13 When something goes wrong

WHAT YOU SEE	WHY	WHAT TO DO
A blank white or black page, and you opened `website/index.html`	Browsers will not let a page opened from your hard drive load its own separate files.	Open `Gradient-Peaks.html` instead. That is what it is for.
Stuck on "Building the terrain…" for more than about 20 seconds	The mesh resolution is too high for this computer, or the function is very slow to evaluate.	Reload the page. Drag **Mesh resolution** down to 100 before changing anything else.
The mouse pointer disappeared and you cannot click anything	Nothing is wrong — the program has taken the mouse so it can turn your head.	Press `Esc`.
Typing in the function box toggles settings	Your cursor is not actually inside the box.	Click directly into the box first. While it holds the cursor, letters type normally.
A red message under the function box	The formula could not be read. The message names the character where it failed.	See the mistakes table in Part 5. The previous landscape is still on screen and still usable.
Large parts of the terrain are missing	The function has no real value there — for instance `sqrt` of a negative number.	Nothing to fix. Change the domain if you want a complete surface; the note at the bottom of the screen tells you how much is undefined.
Everything is jerky	The computer's graphics are struggling.	Part 14, in the order given there.
The explorer will not move	**Directional derivative** is on. It freezes the explorer on purpose.	Turn it off with `B`.
The screen is filled with a solid colour	The camera is inside the terrain or hard against a frontier wall.	Press `R` to return to the centre, or `3` to fly out as the drone.
"WebGL is not available" or similar	Hardware graphics are switched off in the browser.	In Chrome or Edge: Settings → System → turn on *Use graphics acceleration when available*, then restart the browser. On very locked-down machines, ask IT.

14 Making it run faster on an old computer

Change these in order and stop as soon as it feels smooth. The first two cost you almost nothing.

1. **Turn off Shadows** — if it is on, this is the biggest single saving.
2. **Decoration density to 0.3, or to 0** — the scenery is the most expensive part of the picture and none of the mathematics depends on it.
3. **Mesh resolution to 150, then 100** — the surface gets slightly more angular close up, but every number stays exactly as accurate.
4. **Make the browser window smaller** — cost grows with the number of pixels, so a half-size window is roughly twice as fast.
5. **Turn off Level curves** while walking, and back on when you need them.

> Accuracy is never what you are trading away
>
> Every setting in this part changes only how the landscape is *drawn*. The derivatives, the gradient and the optimum are computed from the formula itself, so the numbers on screen are identical at the lowest settings and the highest.

15 What is inside, and licensing

For colleagues and for IT departments who want to know what they are being asked to allow.

It is an ordinary web page

No installer, no background service, no administrator rights. It never sends anything anywhere: it has no network code at all beyond loading its own files. Nothing is collected, stored or transmitted.

Nothing typed by a student is executed as code

The formula box is read by a purpose-built mathematical parser. It can produce a number and nothing else — there is no `eval` anywhere in the program. A student cannot type something into the function box that makes the program do anything other than draw a surface.

The mathematics is honest

All three axis scales start at 1, so one unit of `x`, of `y` and of `z` are the same length and the slope you see is the slope the program reports. (Moving any of the three dials deliberately breaks that, which is why they are labelled as picture-only and start at 1.) The neighbourhood, and every arrow drawn in it, is measured as arc length over the surface — the length the ambient metric assigns to the path — while the derivatives reported are the ordinary partial derivatives, unaffected. The optimum is found by scanning the feasible set and then refining the best candidates, and every trial point must satisfy the constraints — which is why a boundary maximum is found correctly.

Licence

The program is released under the MIT licence: use it, change it, and give it to your students freely. It uses one external library, **three.js** (also MIT, © three.js authors), whose licence text travels with the files in `website/vendor/`.

Appendix A — Keyboard reference card

One page. Print it and leave it next to the projector.

KEY	DOES
`W` `A` `S` `D`	Walk or fly
`Shift`	Hold to move faster
`Space` / `Ctrl`	Drone altitude, up / down
mouse	Look around (click the scene first)
wheel	Camera zoom — the explorer's size does not change
`⌘`/`⊞`/`Alt` + wheel	The explorer's size (the zoom-in ruler)
`⌘`/`⊞`/`Alt` + `W` `A` `S` `D` or arrows	Look around without the mouse
`Esc`	**Release the mouse pointer**
`Tab`	Hide or show the control panel
`1` / `2` / `3`	First person / third person / drone
`T`	Look straight down (map view)
`R`	Return to the centre
`I`	Direction indicator (the globe, top right)
`C`	Level curves
`M`	Colour the surface by height
`N`	Coordinate grid on the surface
`J`	The level curve you are standing on
`K`	Its tangent line, projected on the surface
`F`	Frontier walls of the feasible set
`G`	Make everything outside translucent
`H`	Highlight the neighbourhood disc
`X`	🟦 $\partial f/\partial x$ arrow
`Y`	🟥 $\partial f/\partial y$ arrow
`V`	🟦 Gradient ∇f
`B`	🟨 Directional derivative (freezes the explorer)
`P`	Tangent plane
`O`	Show the optimum

`Enter`	In a text box: apply what you typed

On a phone or tablet: drag on the left half of the screen to walk, drag on the right half to look around.

Appendix B — Every file you received

FILE	WHAT IT DOES
`Gradient-Peaks.html`	The whole program in one file. Double-click to run.
`Gradient-Peaks-Guide.pdf`	This guide.
`README.txt` / `LEEME.txt`	A one-page summary, in English and Spanish.
`website/js/i18n.js`	The English and Spanish text, and the language switch.
`website/index.html`	The page structure and the control panel.
`website/css/style.css`	How the panel and readouts look.
`website/js/mathexpr.js`	Reads what you type into a mathematical expression.
`website/js/field.js`	Converts between mathematical coordinates and metres; computes gradients.
`website/js/terrain.js`	Builds the surface, its colours, the water and the frontier walls.
`website/js/decor.js`	Scatters the trees, rocks, grass and snow.
`website/js/analysis.js`	Level curves, derivative arrows, tangent plane, and the optimiser.
`website/js/player.js`	The explorer, the drone, and the three cameras.
`website/js/main.js`	Puts the scene together and wires up the controls.
`website/vendor/`	three.js, the 3D graphics library, with its licence.
`website/sw.js`	Lets an installed copy work offline.
`website/manifest.webmanifest`	Tells a phone the name and icon to use when installing.
`tools/build-standalone.mjs`	Optional. Rebuilds `Gradient-Peaks.html` if you change the source.

Gradient Peaks — built for teaching the geometry of functions of two variables.
Everything in this guide was checked against the running program.